

```
import kagglehub
path = kagglehub.dataset_download("rakeshrau/social-network-ads")
```

Using Colab cache for faster access to the 'social-network-ads' dataset.

adding lib

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score, precision_score, f1_score
```

reading data

```
df=pd.read_csv(path+"/Social_Network_Ads.csv")
df
```

	User ID	Gender	Age	EstimatedSalary	Purchased	
0	15624510	Male	19	19000	0	
1	15810944	Male	35	20000	0	
2	15668575	Female	26	43000	0	
3	15603246	Female	27	57000	0	
4	15804002	Male	19	76000	0	
...	...	...	...	...	...	
395	15691863	Female	46	41000	1	
396	15706071	Male	51	23000	1	
397	15654296	Female	50	20000	1	
398	15755018	Male	36	33000	0	
399	15594041	Female	49	36000	1	

400 rows × 5 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.describe()
```

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   User ID         400 non-null   int64
1   Gender          400 non-null   object
2   Age             400 non-null   int64
3   EstimatedSalary 400 non-null   int64
4   Purchased       400 non-null   int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

Preprocessing

```
x=df.drop("Purchased",axis=1)
y=df["Purchased"]
```

```
x,y
```

```
(
  User ID  Gender  Age  EstimatedSalary
0  15624510   Male   19             19000
1  15810944   Male   35             20000
2  15668575  Female   26             43000
3  15603246  Female   27             57000
4  15804002   Male   19             76000
..      ...    ...    ...             ...
```

```

395 15691863 Female 46      41000
396 15706071  Male 51      23000
397 15654296 Female 50      20000
398 15755018  Male 36      33000
399 15594041 Female 49      36000

```

```
[400 rows x 4 columns],
```

```
0      0
```

```
1      0
```

```
2      0
```

```
3      0
```

```
4      0
```

```
..
```

```
395    1
```

```
396    1
```

```
397    1
```

```
398    0
```

```
399    1
```

```
Name: Purchased, Length: 400, dtype: int64)
```

```
x=np.array(x)
```

```
y=np.array(y)
```

```
scaler=StandardScaler()
```

```
num_col=df.select_dtypes(include=['float64','int64']).columns
```

```
df[num_col]=scaler.fit_transform(df[num_col])
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y, train_size=0.8, random_state=42)
```

Training the model

```
knn=KNeighborsClassifier(n_neighbors=5)
```

```
# Convert to DataFrame for easier manipulation
```

```
x_train_df = pd.DataFrame(x_train, columns=['User ID', 'Gender', 'Age', 'EstimatedSalary'])
```

```
x_test_df = pd.DataFrame(x_test, columns=['User ID', 'Gender', 'Age', 'EstimatedSalary'])
```

```
# Drop 'User ID' column as it's an identifier and not a feature
```

```
x_train_df = x_train_df.drop('User ID', axis=1)
```

```
x_test_df = x_test_df.drop('User ID', axis=1)
```

```
# One-hot encode 'Gender' column
```

```
x_train_df = pd.get_dummies(x_train_df, columns=['Gender'], drop_first=True)
```

```
x_test_df = pd.get_dummies(x_test_df, columns=['Gender'], drop_first=True)
```

```
# Ensure test set has the same columns as train set (important if a category is missing in test data)
```

```
train_cols = x_train_df.columns
```

```

test_cols = x_test_df.columns
missing_in_test = set(train_cols) - set(test_cols)
for c in missing_in_test:
    x_test_df[c] = 0
x_test_df = x_test_df[train_cols] # Reorder columns to match train

# Scale numerical features ('Age', 'EstimatedSalary')
# Instantiate StandardScaler inside this cell to avoid re-using previously fitted scaler on wrong data
scaler_new = StandardScaler()
numerical_cols = ['Age', 'EstimatedSalary']

x_train_df[numerical_cols] = scaler_new.fit_transform(x_train_df[numerical_cols])
x_test_df[numerical_cols] = scaler_new.transform(x_test_df[numerical_cols])

# Convert back to numpy array for knn.fit
x_train_processed = x_train_df.values
x_test_processed = x_test_df.values

knn.fit(x_train_processed,y_train)

```

▼ KNeighborsClassifier ⓘ ?

KNeighborsClassifier()

```

knn.predict(x_train_processed)
knn.score(x_train_processed,y_train)

```

0.9125

```

# Predict on the training data
y_train_pred = knn.predict(x_train_processed)
acc_train = accuracy_score(y_train, y_train_pred)
print(f"Training Accuracy: {acc_train:.4f}")

# Predict on the test data
y_test_pred = knn.predict(x_test_processed)
acc_test = accuracy_score(y_test, y_test_pred)
print(f"Test Accuracy: {acc_test:.4f}")

```

Training Accuracy: 0.9125
Test Accuracy: 0.9250

```

rec=recall_score(y_test,y_test_pred)
pre=precision_score(y_test,y_test_pred)
f1=f1_score(y_test,y_test_pred)
print(f"Recall: {rec:.4f}")

```

```
print(f"Precision: {pre:.4f}")  
print(f"F1 Score: {f1:.4f}")
```

```
Recall: 0.9286  
Precision: 0.8667  
F1 Score: 0.8966
```

confusion matrix

```
confusion_matrix(y_test,y_test_pred)
```

```
array([[48,  4],  
       [ 2, 26]])
```